Part B: Genetic Algorithm
Word Count: 1984

1. Introduction

This project uses a Genetic Algorithm to evolve simulated PyBullet creatures for a mountain-climbing task. Each creature is represented by a genome controlling body structure and motor behaviour, including link properties, joints, waveform type, amplitude, and frequency. The aim was to improve climbing through selection, crossover, mutation, and elite preservation.The experiments tested how GA settings and encoding choices affected performance. Basic experiments changed population size, genome size, and simulation time. Advanced experiments tested motor-only mutation and hybrid motor output. Finally, an exceptional extension tested the same GA on hill and valley landscapes. Fitness was measured using a climb-focused scoring function and anti-cheat checks to reward genuine contact-based climbing rather than flying or leaving the arena.

2. System Integration

2.1 Mountain Environment Integration

The PyBullet simulation was changed from a simple flat-floor movement task into a controlled mountain-climbing environment. In **simulation.py**, each creature is evaluated inside a fixed arena created using **cw_envt.make_arena()**, and the original **gaussian_pyramid.urdf** terrain is loaded as the main mountain landscape. The simulation also supports alternative landscape files through **landscape_urdf**, allowing the same evaluation pipeline to be reused later for hill and valley experiments

```
# landscape selection
# default is the original gaussian pyramid mountain
# ga.py can replace this with hill.urdf or valley.urdf for the extension
landscape_urdf = getattr(self, "landscape_urdf", "gaussian_pyramid.urdf")
landscape_z = getattr(self, "landscape_z", -1)


# place the selected landscape in the centre of the arena
landscape_position = [0, 0, landscape_z]
landscape_orientation = p.getQuaternionFromEuler([0, 0, 0])


# load the selected landscape as a fixed object
p.loadURDF(
    landscape_urdf,
    landscape_position,
    landscape_orientation,
    useFixedBase=True,
    physicsClientId=pid
)
```

*Code Snippet 1: Landscape loading in **simulation.py***

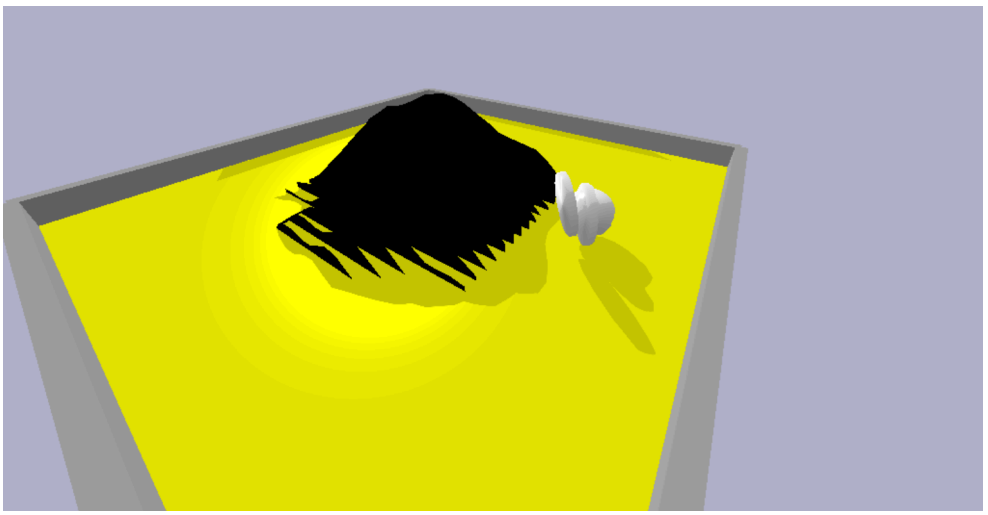


Image 1: PyBullet mountain environment

2.2 Creature Start Position

The creature is spawned near the base of the mountain instead of being dropped randomly into the arena. In the simulation, the start position is set to [5.5, 0.0, 1.0], and the creature is allowed to settle for 120 simulation steps before fitness tracking begins. This reduces unfair early launch behaviour and makes the task focus more on movement towards the slope and contact-based climbing.

```
# start the creature at the bottom/side of the mountain
start_pos = [5.5, 0.0, 1.0]
start_orn = [0, 0, 0, 1]


# place the creature at the starting position
p.resetBasePositionAndOrientation(
    cid,
    start_pos,
    start_orn,
    physicsClientId=pid
)


# clear any starting movement so each creature begins fairly
p.resetBaseVelocity(
    cid,
    linearVelocity=[0, 0, 0],
    angularVelocity=[0, 0, 0],
    physicsClientId=pid
)
```

*Code Snippet 2: Start position and settling period in **simulation.py***

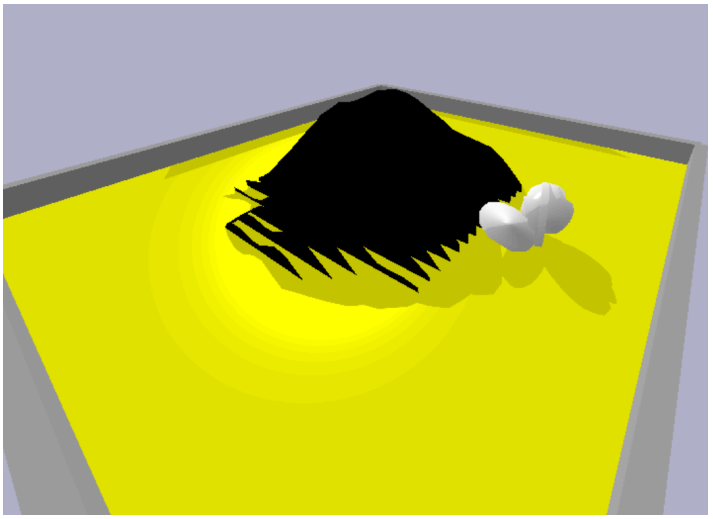


Image 2: Creature starting near the mountain base

2.3 Anti-Cheat System

An anti-cheat system was added to prevent unrealistic behaviours from gaining high fitness. It uses three checks. The surface-contact check uses PyBullet contact points to detect whether the creature is touching the floor, arena wall, or terrain. This information supports both flying detection and fitness tracking. The flying check then uses this contact result with height: if the creature rises above **5.5** and has no contact for **120 steps**, it is marked as cheated and that creature's evaluation stops. A separate boundary check in **simulation.py** also stops creatures that leave the sandbox limits. Together, these checks ensure the GA rewards genuine contact-based climbing rather than flying, bouncing, or escaping the arena.

```python
# Anti-cheat check 1:
# Checks whether the creature is touching the arena, floor, wall,
# or mountain using PyBullet contact points.
def check_touching_surface(self, robot_id, physics_client_id):

    # Get every contact point involving this creature
    contact_points = p.getContactPoints(
        bodyA=robot_id,
        physicsClientId=physics_client_id
    )


    # go through each detected collision
    for contact in contact_points:
        # Id of the object the creature is touching
        other_body = contact[2]


        # ignore collisions with itself
        if other_body != robot_id:
            return True


    # no valid surface contact found
    return False
```

*Code Snippet 3: Anti-cheat touching surface in **anti-cheat.py***

```python
# Anti-cheat check 2:
# Detects flying when the creature is high above the arena or mountain
# and has not touched any surface for a sustained period.
def check_flying(self, current_pos, touching_surface):


    # increase the counter while nothing is being touched
    if not touching_surface:
        self.no_contact_counter += 1
    else:
        # reset counter once contact is made again
        self.no_contact_counter = 0


    # flag the creature if it stays too high for a long time
    if current_pos[2] > self.height_limit and self.no_contact_counter >= self.no_contact_limit:
        self.flying_detected = True


        # Save why it was flagged
        self.reason = "Flying detected: creature is high in the air without surface contact"


        return True


    # flying conditions were not met
    return False
```

*Code Snippet 4:Anti-cheat check flying in **anti-cheat.py***

```python
# run the anti-cheat checks during the real ga evaluation
try:
    flying_detected, touching_surface = anti_cheat.update(
        cid,
        pos,
        pid
    )
except Exception:
    cr.mark_cheated("Anti-cheat check failed", step)
    return


# update the creature's fitness tracking using the surface-contact result
cr.update_position(pos, touching_surface)


# flying anti-cheat check
# if the creature is too high and not touching a surface for too long,
# it is marked as invalid and its fitness will be penalised
if flying_detected:
    cr.mark_cheated(anti_cheat.reason, step)


    print("ANTI-CHEAT TRIGGERED")
    print("Reason:", anti_cheat.reason)
    print("Step:", step)
    print("Position:", pos)
    print("Fitness penalty applied")


    break
```

*Code Snippet 5:Anti-cheat integration in **simulation.py***

2.4 Re-Simulation

The GA training file was kept focused on evolving creatures and saving elite genomes. Separate re-simulation scripts were then used for code modularization to validate the saved elite csv files after training. Each elite was loaded and tested again using the same PyBullet environment, fitness function, and anti-cheat checks. This meant the reported result came from the best validated elite across the whole run, not simply the final generation. The basic evaluator recorded the best elite file, generation, fitness, and anti-cheat status. Separate evaluators were also used for motor-control and landscape experiments, so the correct hybrid motor mode or terrain was applied during testing. This made the final results more reliable and easier to compare.

```
# re test every saved elite file.
for index, elite_file in enumerate(elite_files):
    generation = get_generation_number(elite_file)

    try:
        result = evaluate_elite_file(elite_file)

        fitness = result["fitness"]
        cheated = result["cheated"]
        reason = result["anti_cheat_reason"]
        cheat_step = result["anti_cheat_step"]

        # store the result for the final generation elite.
        if elite_file == final_elite_file:
            final_result = result

        # update the best elite if this one has a higher fitness.
        if best_fitness is None or fitness > best_fitness:
            best_fitness = fitness
            best_elite_file = elite_file
            best_generation = generation
            best_cheated = cheated
            best_reason = reason
            best_cheat_step = cheat_step

        print(
            "Checked:",
            os.path.basename(elite_file),
            "| Generation:", generation,
            "| Fitness:", fitness,
            "| Anti-cheat:", cheated
        )
```

Code Snippet 6: Separate Elite re-simulation

3. Fitness Function

3.1 Position Tracking

The original movement task only measured how far a creature moved from its start position. This was not suitable for mountain climbing because a creature could score highly by moving sideways, jumping away, or being launched into the air. Therefore, **update_position()** was changed to track climbing-related signals, including distance from the mountain peak, highest surface-contact height, best height near the slope, continuous upward movement, time near the top, and airborne steps. Most rewards are only updated when the creature is touching a surface, encouraging the GA to evolve real climbing behaviour instead of physics exploits.

```python
## i did this part
    def update_position(self, pos, touching_surface=True):
        x = pos[0]
        y = pos[1]
        z = pos[2]

        distance_from_peak = np.sqrt(x**2 + y**2)

        # First position after settling becomes the official start point.
        if self.start_position is None:
            self.start_position = pos
            self.last_position = pos
            self.prev_position = pos
            self.max_height = z
            self.min_distance_from_peak = distance_from_peak
            return

        self.last_position = pos

        if touching_surface:
            self.air_streak = 0

            # Track closest distance to the mountain peak while touching the ground/mountain.
            if self.min_distance_from_peak is None:
                self.min_distance_from_peak = distance_from_peak
            elif distance_from_peak < self.min_distance_from_peak:
                self.min_distance_from_peak = distance_from_peak

            # Track highest point reached while touching the surface.
            if z > self.max_height:
                self.max_height = z

            # Track best height only when creature is near the mountain slope.
            if distance_from_peak < 5.5:
                if z > self.best_height_near_mountain:
                    self.best_height_near_mountain = z

            # Reward continuous upward movement.
            if self.prev_position is not None:
                height_change = z - self.prev_position[2]
                if height_change > 0:
                    self.uphill_score += height_change

            # Reward staying close to the top.
            if distance_from_peak < 1.5 and z > 3.0:
                self.top_stay_steps += 1

        else:
            self.air_steps += 1
            self.air_streak += 1

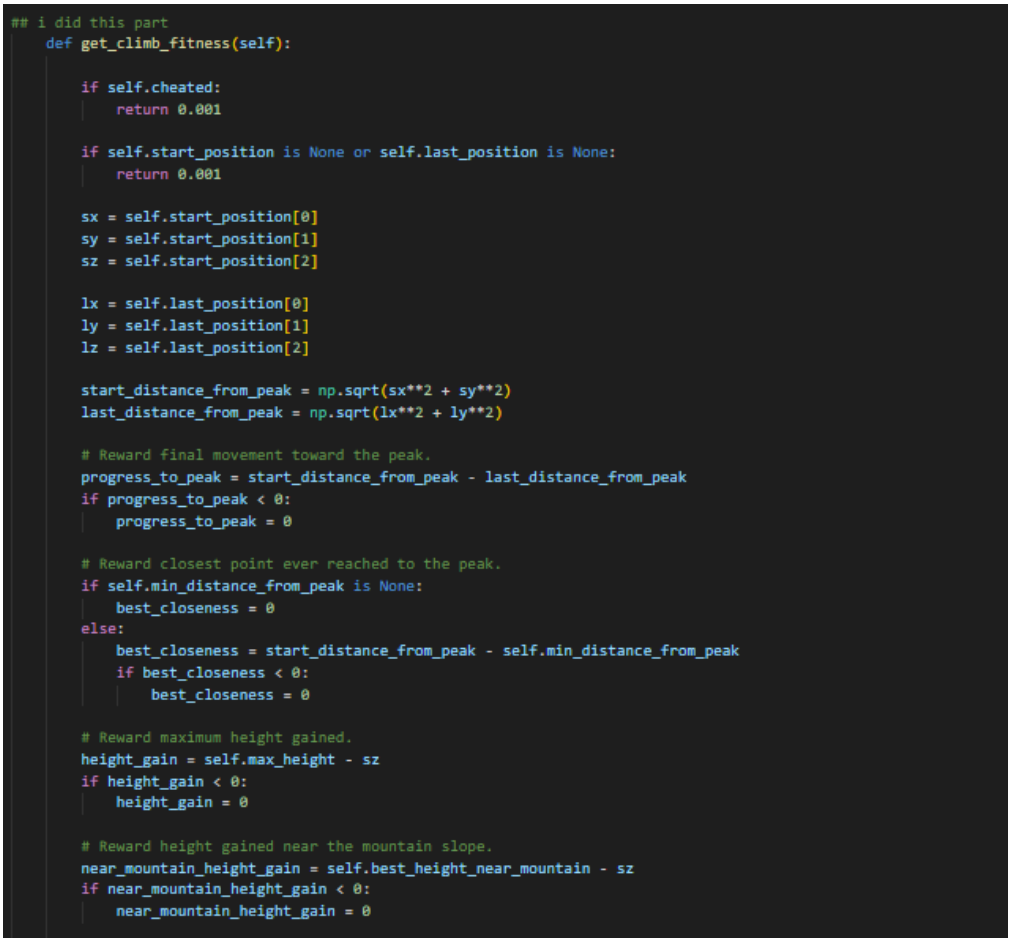
            if self.air_streak > self.longest_air_streak:
                self.longest_air_streak = self.air_streak

        self.prev_position = pos
```

Code Snippet 7: Position tracking used to record mountain climbing behaviour.

3.2 Climb Fitness Design

The fitness function was redesigned to match the goal of reaching higher positions on the mountain without cheating. Distance alone was not reliable because moving far does not prove the creature climbed. The final fitness therefore uses a weighted score based on progress towards the peak, closest distance reached, maximum height gain, height gain near the slope, final height gain, uphill movement, and time near the top. The strongest reward is given to height gained near the mountain because it best represents real climbing. Penalties are also included: cheated creatures receive a fitness of **0.001,** and excessive airborne behaviour reduces the score. This makes the GA search for contact-based climbing instead of unrealistic bouncing or flying.



```python
## i did this part
    def get_climb_fitness(self):

        if self.cheated:
            return 0.001

        if self.start_position is None or self.last_position is None:
            return 0.001

        sx = self.start_position[0]
        sy = self.start_position[1]
        sz = self.start_position[2]

        lx = self.last_position[0]
        ly = self.last_position[1]
        lz = self.last_position[2]

        start_distance_from_peak = np.sqrt(sx**2 + sy**2)
        last_distance_from_peak = np.sqrt(lx**2 + ly**2)

        # Reward final movement toward the peak.
        progress_to_peak = start_distance_from_peak - last_distance_from_peak
        if progress_to_peak < 0:
            progress_to_peak = 0

        # Reward closest point ever reached to the peak.
        if self.min_distance_from_peak is None:
            best_closeness = 0
        else:
            best_closeness = start_distance_from_peak - self.min_distance_from_peak
            if best_closeness < 0:
                best_closeness = 0

        # Reward maximum height gained.
        height_gain = self.max_height - sz
        if height_gain < 0:
            height_gain = 0

        # Reward height gained near the mountain slope.
        near_mountain_height_gain = self.best_height_near_mountain - sz
        if near_mountain_height_gain < 0:
            near_mountain_height_gain = 0
```

```
# Reward final height so the creature does not just touch high point and fall away.
final_height_gain = lz - sz
if final_height_gain < 0:
    final_height_gain = 0

# Small reward for staying near the top.
stay_bonus = self.top_stay_steps * 0.01

# Penalise excessive air time.
air_penalty = self.air_steps * 0.001

fitness = (
    progress_to_peak * 5.0 +
    best_closeness * 8.0 +
    height_gain * 20.0 +
    near_mountain_height_gain * 40.0 +
    final_height_gain * 15.0 +
    self.uphill_score * 10.0 +
    stay_bonus -
    air_penalty
)

# Strong penalty if airborne for more than 2 seconds.
# 480 simulation steps is about 2 seconds at 240 FPS.
if self.longest_air_streak > 480:
    fitness = fitness * 0.1

if fitness <= 0:
    fitness = 0.001

return fitness
```

Code Snippet 8: Weighted climb fitness function used to reward genuine climbing.

4. Basic GA Experiments

The first experiments tested three main GA settings: population size, genome size, and simulation time. In each test, only one variable changed while the others stayed constant. Each setting was tested over five runs, and the results were compared using Highest Best Fitness and Average Best Fitness. Standard deviation was also included to show variation across the five runs. Larger standard deviation values indicate that the setting was less consistent, even when the average fitness was high.

A limitation of these comparisons is that larger populations and longer simulations also increase the total computational work. For example, population size 40 evaluates more creatures than population size 10, so some improvement may come from having more search opportunities rather than the setting alone.

4.1 Population Size

Population size was tested because it affects the amount of variation available in each generation. A larger population gives the GA more creatures to select from, which increases the chance of finding a strong climbing behaviour.

Testing 1 : Population Size

Run ID	Population Size	Initial Genome Size	Mutation Rate	Number of Generation	Simulation Rate	Highest Best Fitness (5 runs)	Average Best Fitness (5 runs)	Standard Deviation (5 runs)
1	10	3	0.1	100	2400	213.683	171.0538	26.95322427
2	20	3	0.1	100	2400	319.861	235.6926	58.87924326
3	30	3	0.1	100	2400	942.061	380.4962	314.3090183
4	40	3	0.1	100	2400	1347.626	646.2178	450.9412954

Image 3: Testing 1 table: Population Size

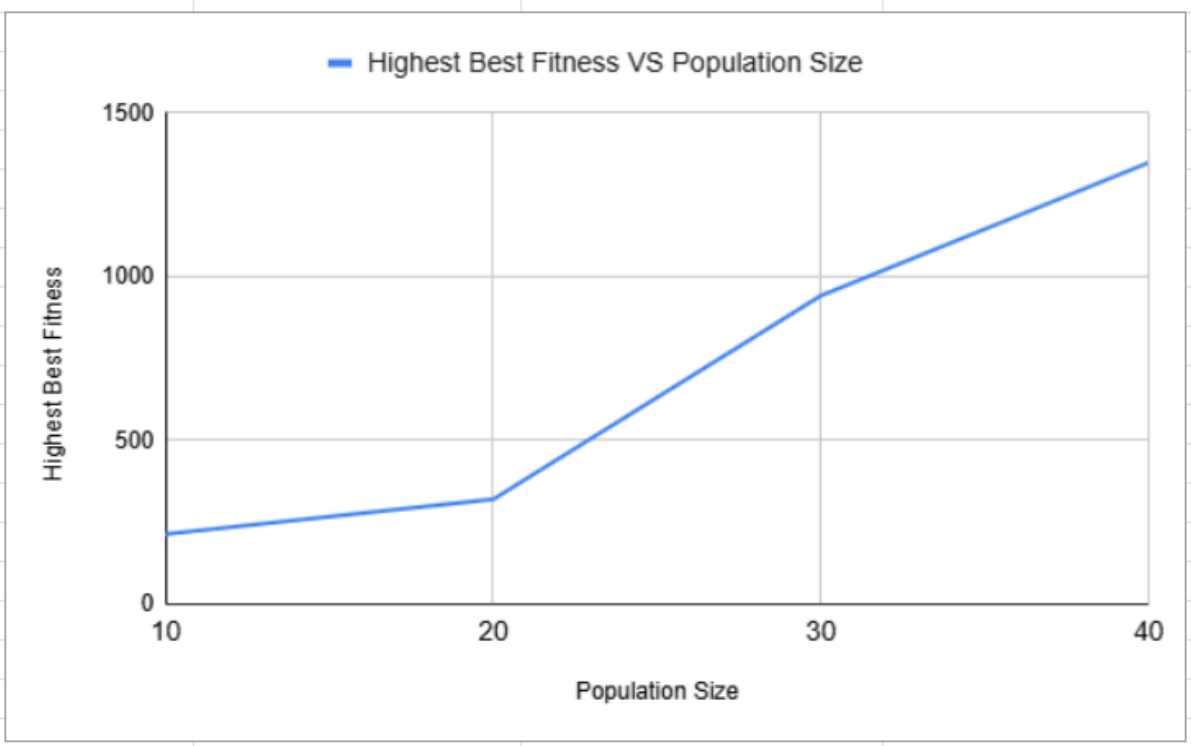


Image 4 : Highest Best Fitness vs Population Size Graph

The highest best fitness increased from **213.683** at population size 10 to **1347.626** at population size 40. This shows that larger populations made it more likely for the GA to discover at least one high-performing climber.

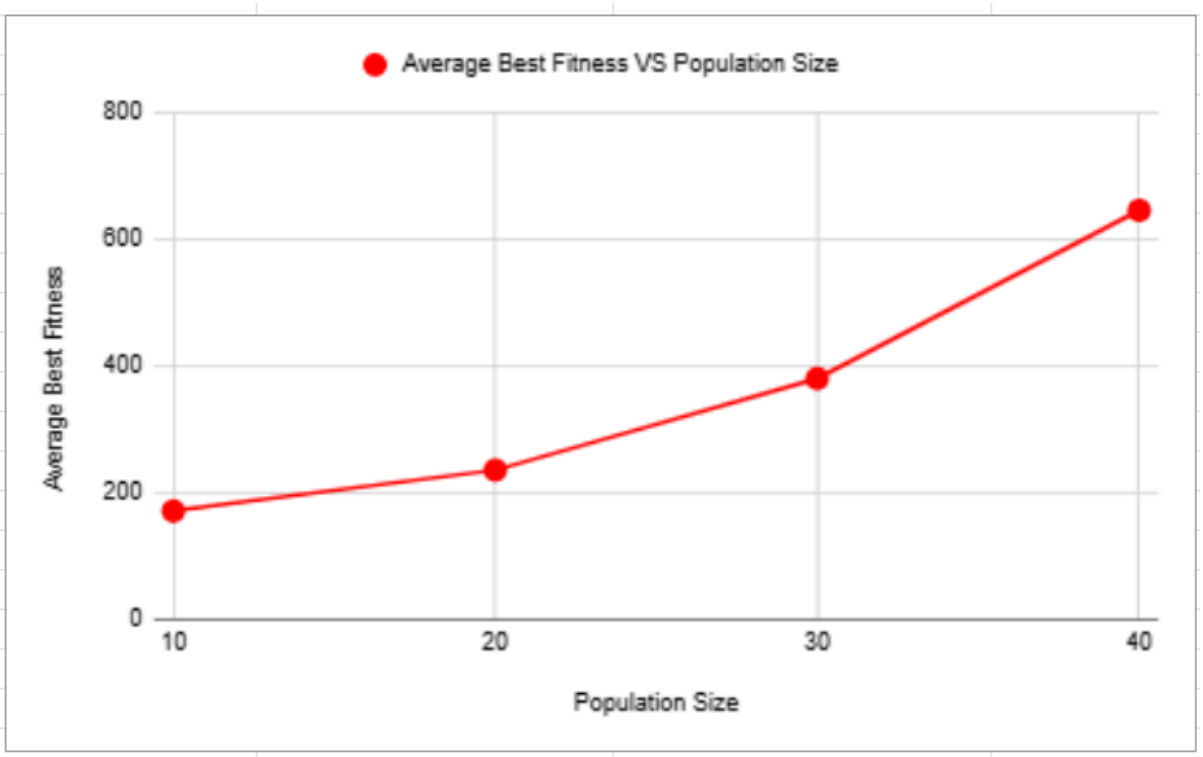


Image 5: Average Best Fitness vs Population Size Graph

The average best fitness also increased from **171.0538** to **646.2178**. This is important because it shows that the improvement was not only caused by one good run. Overall, population size 40 gave the strongest performance among the tested values.

4.2 Genome Size

Genome size was tested because it controls how much genetic information is available to build the creature. A larger genome can allow more complex body structures and movement patterns, but it also increases the search space.

Testing 2 : Genome Size

Run ID	Population Size	Initial Genome Size	Mutation Rate	Number of Generation	Simulation Rate	Highest Best Fitness (5 runs)	Average Best Fitness (5 runs)	Standard Deviation (5 runs)
1	10	3	0.1	100	2400	213.683	171.0538	26.95322427
2	10	6	0.1	100	2400	322.475	244.2472	63.42682135
3	10	9	0.1	100	2400	1407.833	671.0416	584.5109209

Image 6: Testing 2 table: Genome Size

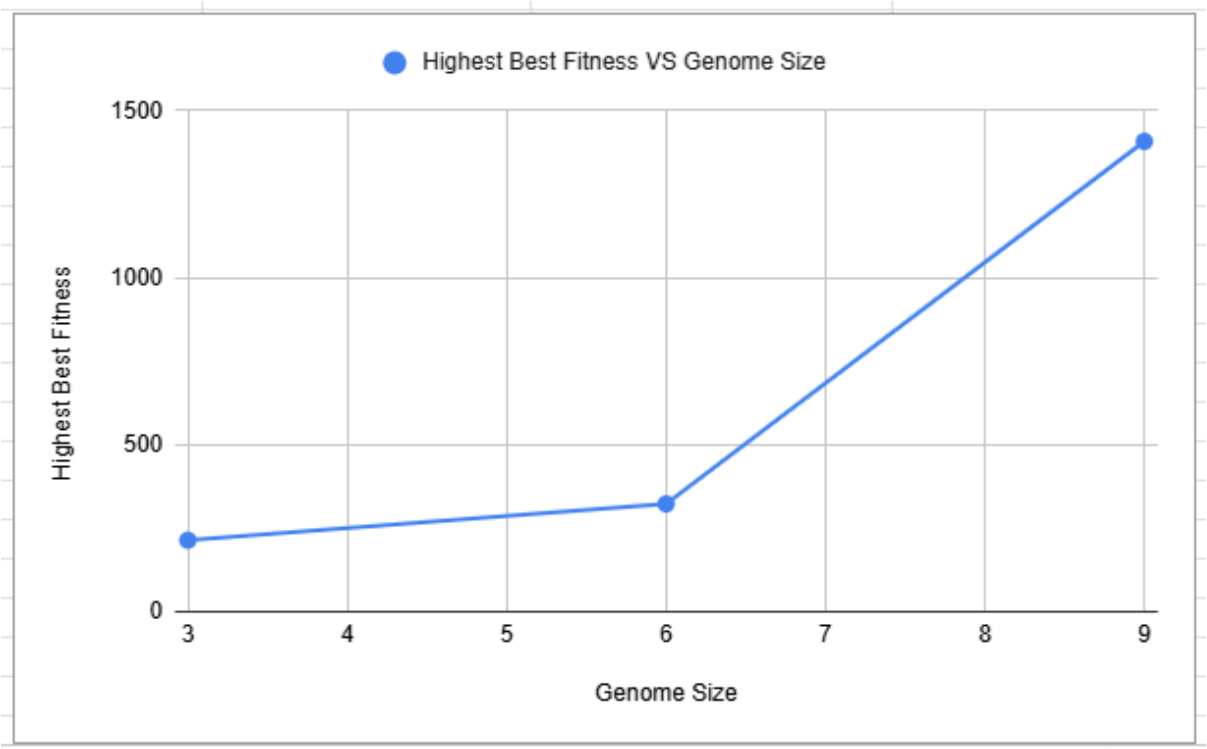


Image 7: Highest Best Fitness vs Genome Size Graph

The highest best fitness increased from **213.683** with 3 genes to **1407.833** with 9 genes. This suggests that the larger genome gave the GA more opportunity to evolve useful body and joint structures for climbing.

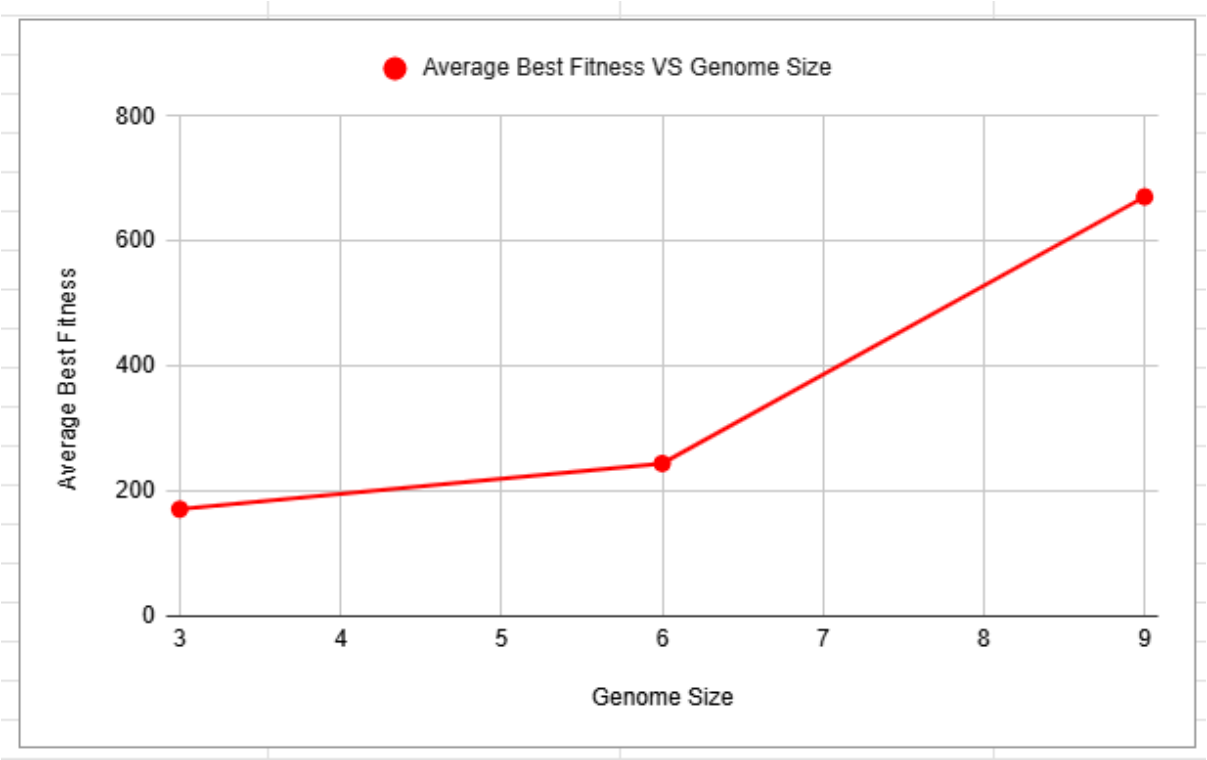


Image 8: Average Best Fitness vs Genome Size Graph

The average best fitness also improved from **171.0538** to **671.0416**. This shows that 9 genes did not only produce one strong outlier, but also gave better performance across the five runs. Therefore, 9 genes was the strongest genome-size setting in this experiment.

4.3 Simulation Time

Simulation time was tested because it controls how long each creature is evaluated. A short simulation may not give enough time to reach the mountain, while a longer simulation gives more opportunity for movement and height gain.

Testing 3 : Time Simulation

Run ID	Population Size	Initial Genome Size	Mutation Rate	Number of Generation	Simulation Rate	Highest Best Fitness (5 runs)	Average Best Fitness (5 runs)	Standard Deviation (5 runs)
1	10	3	0.1	100	2400	213.683	171.0538	26.95322427
2	10	3	0.1	100	3600	904.898	355.1968	312.956016
3	10	3	0.1	100	4800	1343.446	487.4042	479.2377277

Image 9: Testing 3 table: Time Simulation

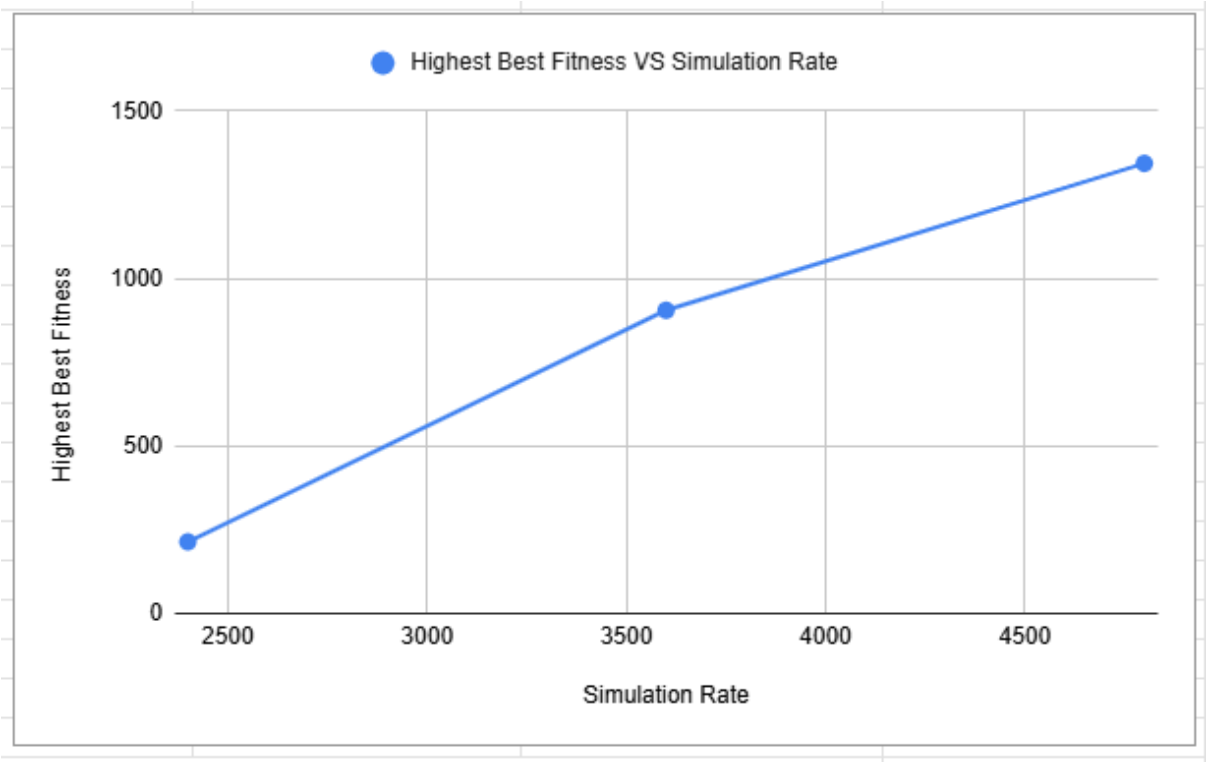


Image 10: Highest Best Fitness vs Simulation Rate Graph

The highest best fitness increased from **213.683** at 2400 steps to **1343.446** at 4800 steps. This suggests that longer simulations helped some creatures show stronger climbing behaviour.

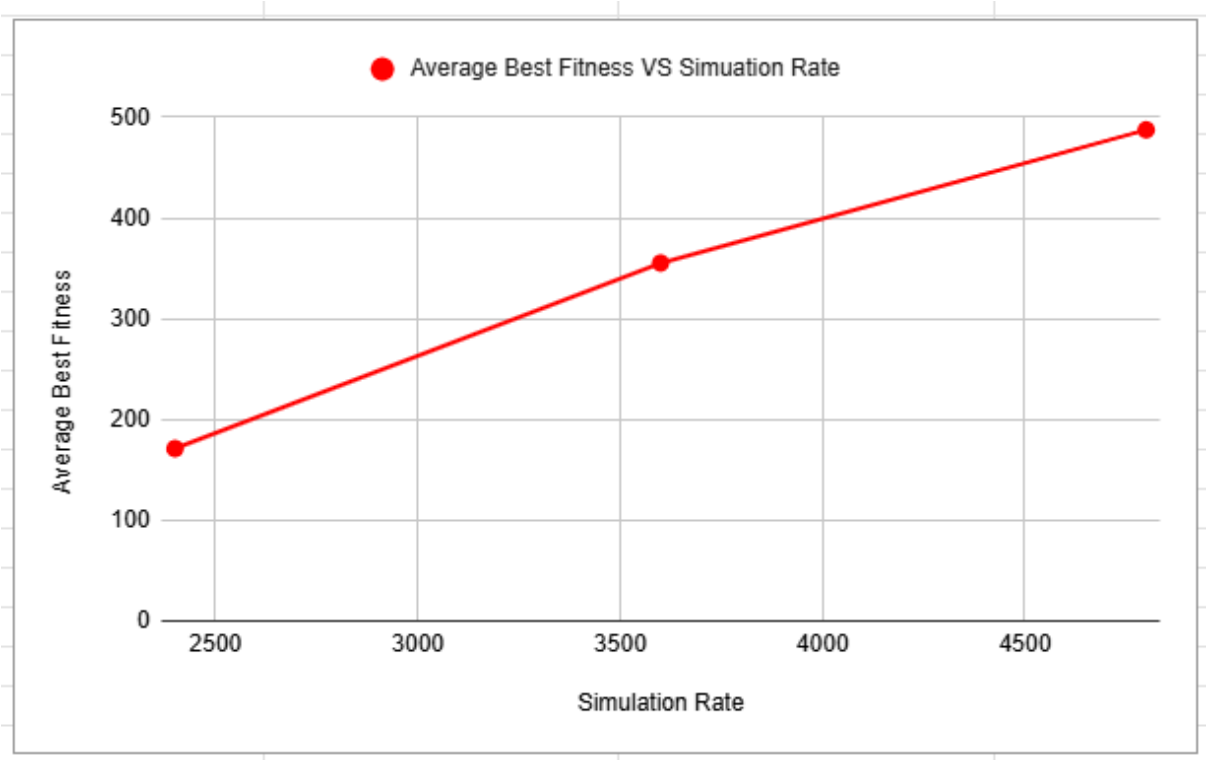


Image 11 : Average Best Fitness vs Simulation Rate

The average best fitness also increased steadily from **171.0538** to **487.4042**. This confirms that longer simulation time improved performance across multiple runs, not just in one best case. Based on these results, 4800 was the strongest simulation-time setting, although it also requires more processing time.

5. Advanced Motor-Control Encoding Experiments

5.1 Baseline and Motor_only

This section tested whether changing motor-control encoding could improve climbing. The **Baseline** mode allowed both body and motor genes to evolve, giving the GA freedom to improve structure and movement together.

The **Motor_only** mode restricted mutation to motor genes only: control waveform, control amplitude, and control frequency. Non-motor genes were skipped, so the experiment mainly tested whether motor tuning alone could improve climbing.

```python
@staticmethod
def point_mutate(genome, rate, amount, motor_only=False):
    # copies the genome so mutation does not directly edit the original object
    new_genome = copy.copy(genome)


    motor_indices = None


    # when motor_only is enabled, only motor-control genes are allowed to mutate
    ##### I did this part ############################
    if motor_only:
        spec = Genome.get_gene_spec()
        motor_indices = {
            spec["control-waveform"]["ind"],
            spec["control-amp"]["ind"],
            spec["control-freq"]["ind"]
        }


    for gene in new_genome:
        for i in range(len(gene)):


            # skip body-related genes when testing motor-only mutation
            if motor_indices is not None and i not in motor_indices:
                continue


            # apply a small random change based on the mutation rate
            if random.random() < rate:
                random_change = (random.random() - 0.5) * 2 * amount
                gene[i] += random_change


            # keep gene values inside the valid 0 to 1 range
            if gene[i] >= 1.0:
                gene[i] = 0.9999


            if gene[i] < 0.0:
                gene[i] = 0.0


    return new_genome
```

Code Snippet 9: Motor-only mutation

5.2 Hybrid Motor Output

The **Hybrid** mode changed how motor output was generated. Instead of choosing only pulse or sine movement, the hybrid version blended both using the waveform value. This gave the creature a smoother range of possible joint movement. The **Motor_Hybrid** mode combines motor-only mutation with hybrid output.

```python
## i did this part
    def get_output(self):
        self.phase = (self.phase + self.freq) % (np.pi * 2)

        pulse_output = 1 if self.phase < np.pi else -1
        sine_output = np.sin(self.phase)

        # Baseline mode: original pulse or sine motor control.
        if not self.hybrid:
            if self.motor_type == MotorType.PULSE:
                return pulse_output
            else:
                return sine_output

        # Hybrid mode: mix pulse and sine instead of choosing only one.
        mixed_output = (1.0 - self.waveform) * pulse_output + self.waveform * sine_output

        # control_amp is scaled small in genome.py, so scale it back up.
        amp_scale = self.amp * 4.0

        # Prevent completely dead motors.
        if amp_scale < 0.2:
            amp_scale = 0.2

        return mixed_output * amp_scale
```

Code Snippet 10: Hybrid motor output formula

5.3 Results of Advanced Motor Controls

Testing 4 : Advanced Motor Controls

Motor Controls	Population Size	Initial Genome Size	Mutation Rate	Number of Generation	Simulation Rate	Highest Best Fitness (5 runs)	Average Best Fitness (5 runs)	Standard Deviation (5 runs)
Baseline	10	3	0.1	100	2400	213.683	171.0538	26.95322427
Motor_only	10	3	0.1	100	2400	186.351	139.175	33.44334914
Hybrid	10	3	0.1	100	2400	489.745	269.5296	124.3474112
Motor_Hybrid	10	3	0.1	100	2400	149.097	120.626	35.9035219

Image 12: Testing 4 table: Advanced Motor Controls

The table compares Baseline, Motor_only, Hybrid, and Motor_Hybrid using the same population size, genome size, mutation rate, generation count, and simulation rate.

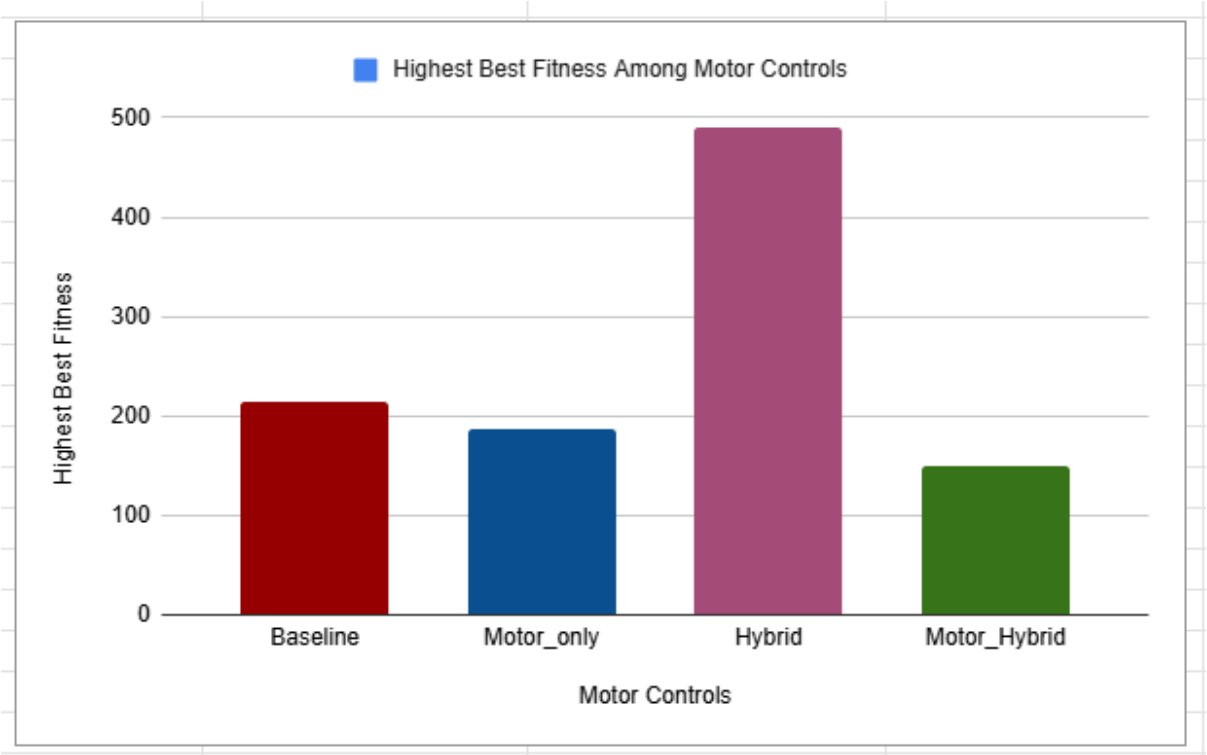


Image 13: Highest Best Fitness Among Motor Controls Graph

The highest best fitness graph shows that **Hybrid** performed best, reaching **489.745**. This is higher than Baseline at **213.683**, Motor_only at **186.351**, and Motor_Hybrid at **149.097**.

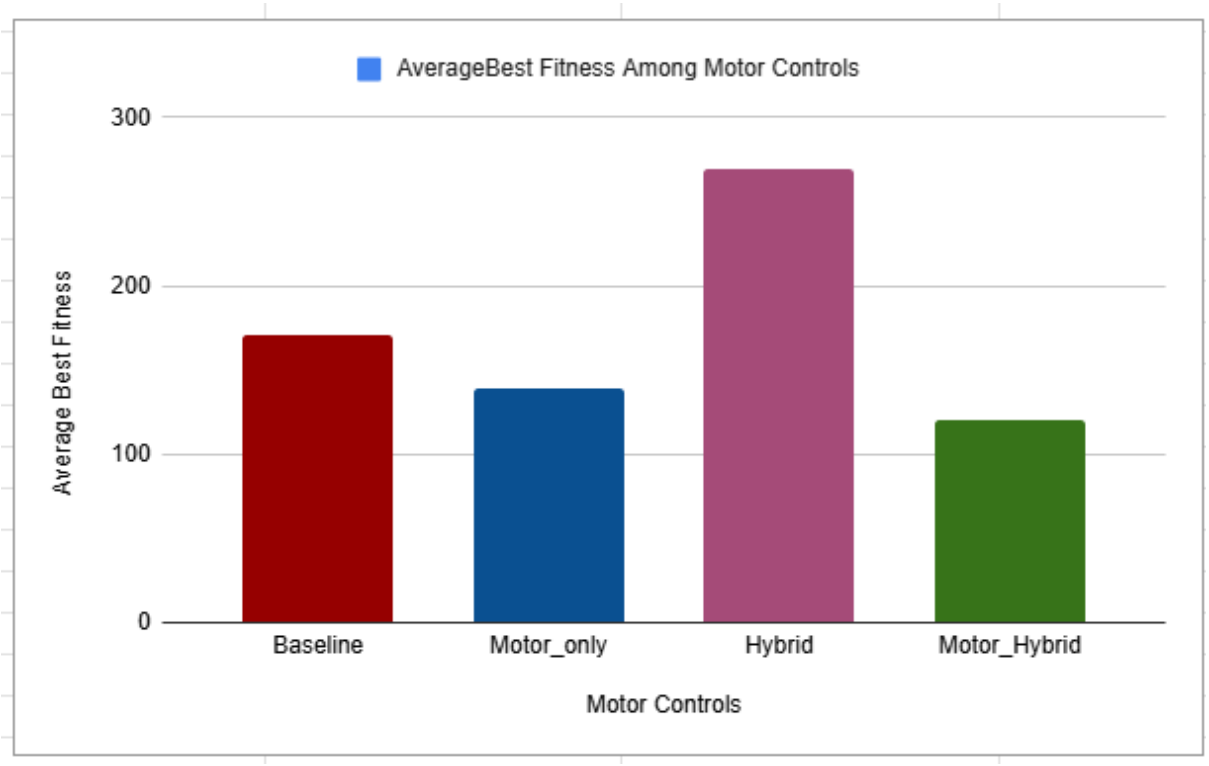


Image 14: Average Best Fitness Among Motor Controls Graph

The average best fitness graph shows the same pattern. **Hybrid** achieved the best average result of **269.5296**, while Baseline reached **171.0538**, Motor_only reached **139.175**, and Motor_Hybrid reached **120.626**.

These results suggest that changing motor output helped more than restricting mutation to motor genes only. Hybrid performed best because blending pulse and sine movement gave more flexible joint control while still allowing body structure to evolve. However, Motor_Hybrid performed worst. This suggests that combining hybrid output with motor-only mutation was too restrictive because the creature could adjust its movement pattern but could not evolve body structures to match the new control style. Therefore, climbing performance depended on co-evolution between body structure and motor control, not motor tuning alone.

6. Exceptional Extension: Different Landscapes

The exceptional extension tested whether the GA could work on different terrain types. The original environment used the Gaussian Pyramid, while two new landscapes, **Hill** and **Valley**, were created in **prepare_shapes.py** and loaded using separate URDF files.

The same settings were used for all landscapes: population size **10**, genome size **3**, mutation rate **0.1**, **100** generations, and simulation time **2400**. This kept the comparison fair because only the environment changed.

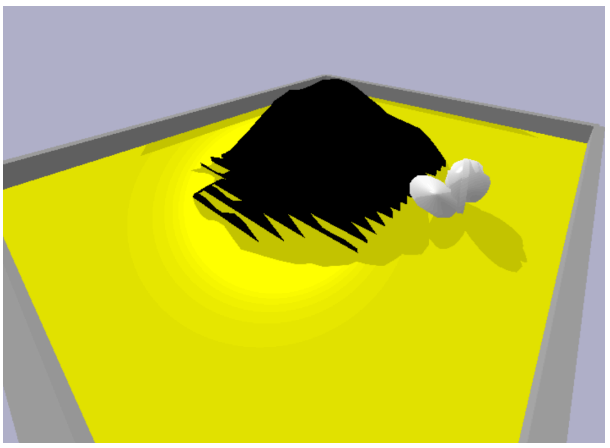


Image 15: screenshots of Guassian Pyramid

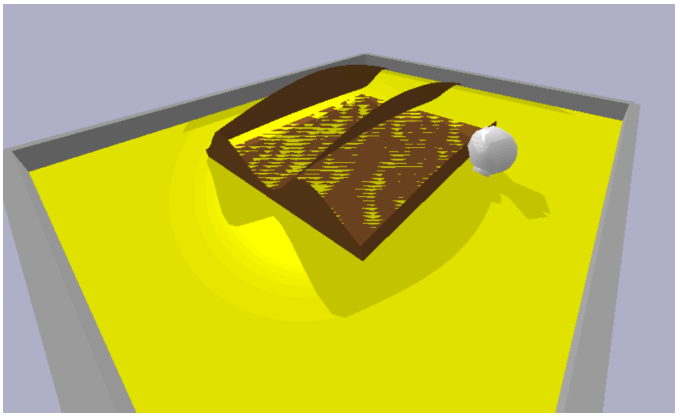


Image 16: screenshots of Valley

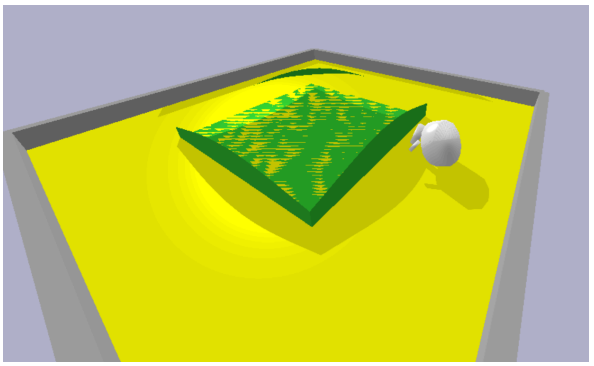


Image 17: screenshots of Hill

Testing 5 : Different Environment

Landscape	Population Size	Initial Genome Size	Mutation Rate	Number of Generation	Simulation Rate	Highest Best Fitness (5 runs)	Average Best Fitness (5 runs)	Standard Deviation (5 runs)
Guassian Pyramid	10	3	0.1	100	2400	213.683	171.0538	26.95322427
Hill	10	3	0.1	100	2400	450.953	261.69	118.0585353
Valley	10	3	0.1	100	2400	225.828	191.6556	25.81253623

Image 18: Testing 5 table: Different Environment

The table shows the results for each landscape using the same GA settings.

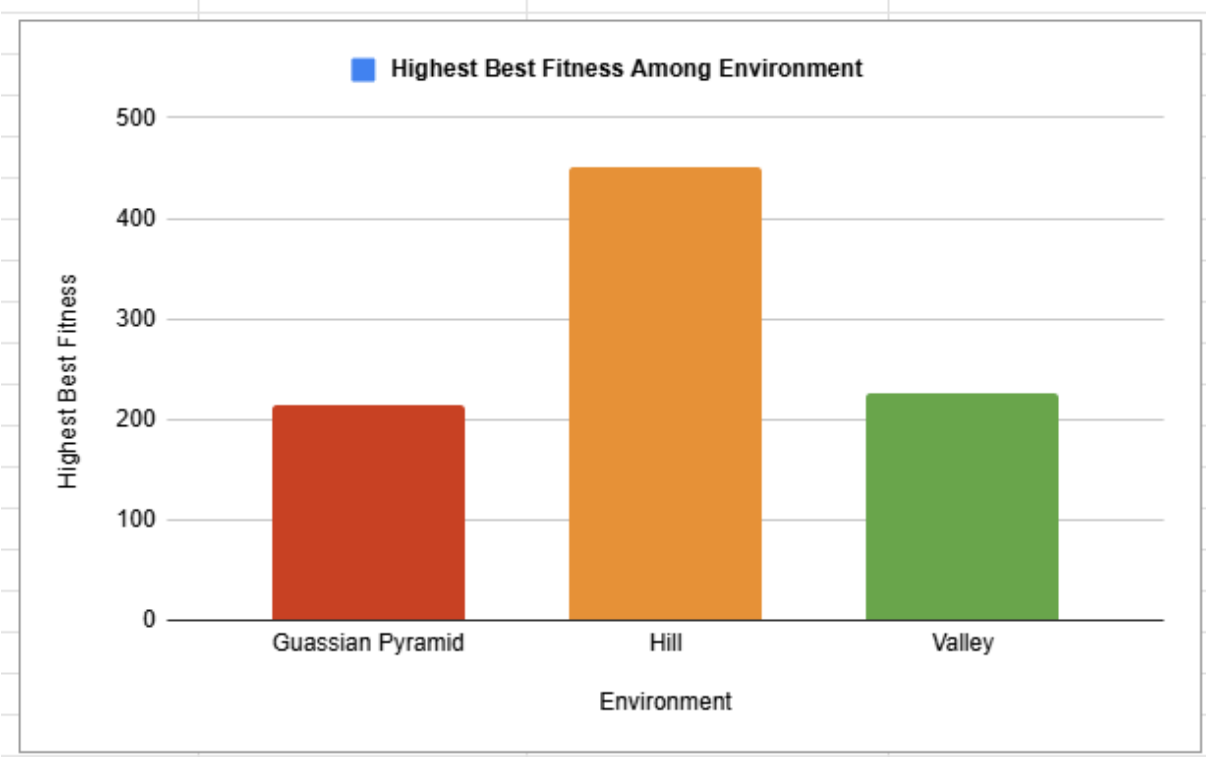


Image 19 : Highest Best Fitness Among Environment Graph

The highest best fitness graph shows that the **Hill** produced the strongest single result, reaching **450.953**. The Gaussian Pyramid reached **213.683**, while the Valley reached **225.828**.

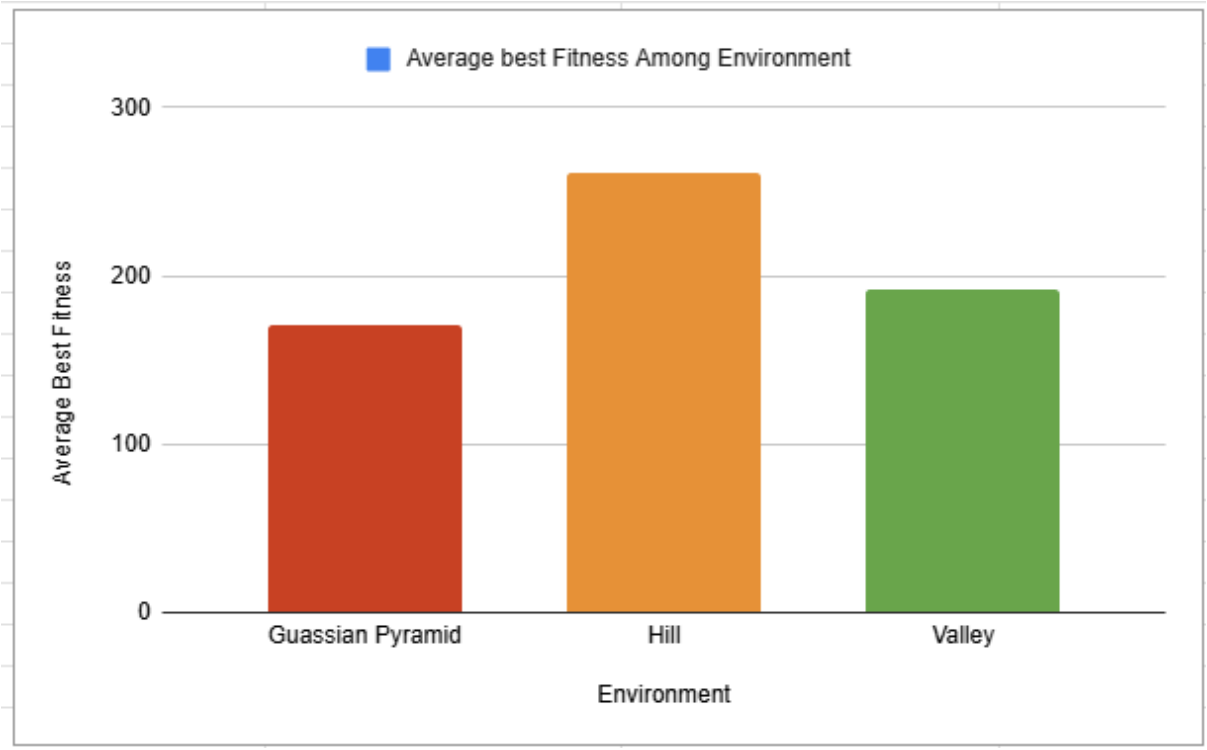


Image 20 : Average Best Fitness Among Environment Graph

The average best fitness graph also shows that the Hill performed best, with **261.69**. The Valley achieved **191.6556**, which was slightly higher than the Gaussian Pyramid at **171.0538**.

These results suggest that the smoother Hill was easier to climb because it provided more gradual and stable contact. The Valley was harder than the Hill because its shape changes the slope direction and requires better balance. Overall, the extension shows that GA performance depends strongly on terrain shape.

7. Conclusion

This project developed a Genetic Algorithm system for evolving PyBullet creatures to perform contact-based mountain climbing. The original movement task was extended by adding a mountain arena, a fixed start position, anti-cheat checks, elite re-simulation, and a climb-focused fitness function. The basic experiments showed that performance improved most with population size **40**, genome size **9**, and simulation time **4800**. The motor-control experiments showed that **Hybrid** output performed best because blending pulse and sine gave more flexible movement. The landscape extension showed that **Hill** achieved the strongest result, while **Valley** was harder but still slightly better than the Gaussian Pyramid. Overall, climbing performance depended on body structure, motor control, simulation time, and terrain shape.